

COSCA vs. HERMES AGENT

Análise Comparativa Técnica — Código a Código

Vantagens de cada sistema, capacidades e veredito

*Preparado para o **Don** — família Cosca*

Emissão: 27 de agosto de 2026

Versão Cosca: v1.5.0 · Versão Hermes: main · Fontes: repositórios oficiais

Sumário

- 1. Sumário Executivo
- 2. Os Dois Projetos em Números
- 3. Fundamentos de Arquitetura (lado a lado)
- 4. Comparação Código-a-Código (tabela de mapeamento)
- 5. Matriz de Capacidades — O que cada um consegue fazer
- 6. Vantagens do Cosca
- 7. Vantagens do Hermes Agent
- 8. Onde cada um perde (gaps honestos)
- 9. Veredicto e Recomendação para o Don
- 10. Apêndice — Metodologia e Fontes

Resumo em uma frase: O **Cosca** é um *cérebro corporativo* — governança, memória semântica curada, segurança fail-closed e um mundo 3D determinístico. O **Hermes Agent** é um *agente de produção* — pronto para rodar em qualquer lugar, com learning loop self-improving, 37 providers e 7 backends de execução, focado em ubiquidade (Telegram/Discord/Web/Desktop).

1. Sumário Executivo

Esta análise compara o **Cosca** (github.com/CoscaAI/cosca — monólito Go) e o **Hermes Agent** (github.com/nousresearch/hermes-agent — Python + TypeScript). O objetivo não é declarar um 'vencedor' — os dois respondem a perguntas diferentes. A comparação é feita **código a código** e **capacidade a capacidade**, de forma honesta, apontando o que cada um faz melhor e onde cada um é exposto.

Natureza dos projetos (a diferença de fundo):

Eixo	Cosca	Hermes Agent
Identidade	Cérebro corporativo / governança de conhecimento. Uma hierarquia operacional (Don -> Kernel -> CEO -> CTO -> Chiefs -> Specialists) com 'cérebro de conhecimento curado'. Foco em verdade auditável e controle .	Agente pessoal de produção para o usuário. Self-improving, multi-plataforma, roda em qualquer lugar (VPS, GPU cluster, serverless). Foco em ubiquidade e autonomia .
Stack	Go 1.26 (uma linguagem, um binário). SQLite puro (modernc), WASM (wazero), Cobra CLI.	Python 3.11 + TypeScript/React. SQLite (FTS5), Node.js, Nix, Docker.
Escala	~25.000 arquivos .go, ~846 MB de fonte, ~127 pacotes em internal/, 97 comandos CLI, 94 endpoints REST, 12 RPCs gRPC.	~10.500 arquivos (4689 .py, 1851 .ts, 1556 .md), 37 providers, 82+118 skills, 7 backends de execução.
Força central	Determinismo zero-LLM: 'a IA propõe, o sistema decide'. Aritmética de gate testável com go test, sem redenção de modelo. Fail-closed absoluto.	Learning loop: cria skills da experiência, melhora durante o uso, busca nas próprias conversas e constrói um modelo de quem você é ao longo das sessões.

Veredito de alto nível: O **Cosca** **vence** em governança, segurança, determinismo, memória semântica curada, auditoria/proveniência e arquitetura de máquina corporativa. O **Hermes Agent** **vence** em prontidão de produção, ubiquidade de interfaces (Telegram/Discord/Web/Desktop), portabilidade de providers, soma de skills prontas e ecossistema self-improving. São **complementares**, não substitutos.

2. Os Dois Projetos em Números

Métrica	Cosca	Hermes Agent
Linguagem primária	Go (Go 1.26) — binário único, tipado, compilado	Python 3.11 (core) + TypeScript/React (UI)
Licença	Proprietária (license.go exige 3 fatores de autenticidade)	MIT (open source)
Tamanho do repo	~25.500 arquivos .go · ~846 MB de fonte · git ~ grande	~10.500 arquivos · 4.689 .py · 1.851 .ts · 1.556 .md
Pacotes/domínios	~127 pacotes em internal/ + API + SDK	agent/, tools/, gateway/, cron/, plugins/, providers/, skills/
Comandos CLI	97 comandos top-level (Cobra)	CLI Python (<code>hermes ...</code>) + TUI Ink
API	94 endpoints REST + gRPC (3 serviços/12 RPCs) + OpenAPI	JSON-RPC 2.0 (TUI/Web/Desktop) + REST + MCP
Providers LLM	~12 nativos (anthropic, openai, google, azure, deepseek, groq, mistral, ollama, local, bedrock, openaicompat, bootstrap)	37 providers em plugins (Nous, OpenAI, Anthropic, Gemini, Bedrock, OpenRouter, Copilot, DeepSeek, xAI...)
Skills	29 em disco (formato .md próprio) + skilleval	82 padrão + 118 optional (formato agentskills.io) — 200 no total
Agentes	61 departments (embeddados) — hierarquia Don->Kernel->CEOs->Chiefs->Specialists	1 agente core + subagentes paralelos (delegate_task)
Backends de execução	Auto-jail (Bubblewrap) + WASM + plugins; sandbox subprocess seccomp	7 backends: local, Docker, SSH, Singularity, Modal, Daytona, Vercel Sandbox
Mundo 3D	world/, worldmodel/, scene/, nodegraph/, gameengine (ECS), procgen, city, vfx, audio, destruction, multiagent — determinístico	Não possui (é agente de texto/tools, não de mundo 3D)
Memória	6 camadas com TTL + FTS5 + embeddings + blockchain/Merkle + RBAC por papel	SQLite FTS5 + memória curada (MEMORY.md/USER.md) + Honcho (modelo de usuário)
Busca	Híbrida FTS5 + vetor int8/16/32 (AVX2) + grafo + re-rank multi-fator + meaning-first	FTS5 + busca semântica em sessões passadas (session_search) + skill search
Cron/automação	cron (scheduler), workflows (dflow determinístico)	cron/ scheduler + entrega em 20+ plataformas

Nota metodológica: os números do Cosca foram medidos diretamente no repositório da família (contagem de arquivos .go, comandos em root.go, endpoints em server.go). Os do Hermes foram medidos no clone oficial (contagem de extensões e leitura dos registros de toolsets/providers). Ambas as medições são aproximadas e refletem o estado do código no momento da análise.

3. Fundamentos de Arquitetura (lado a lado)

3.1 Cosca — 'cérebro corporativo'

O Cosca é organizado como uma **hierarquia operacional** com um **control plane** e um **data plane** bem separados. A IA propõe; o sistema decide; a aritmética de gate é determinística e testável.

```
Entrada (LLM external) -> PROPOSAL -> KERNEL ISOLADO -> APPROVE/DENY/REVIEW -> EXECUCAO
(proposal.go: flow.go validator.go) -> 'a IA aprova, o sistema decide'
CAMERA -> Vision -> Spatial -> World State -> Multi-Agent -> Decision -> Action -> Unreal
(worldmodel/orchestrator.go)
Search = FTS5 + Vector(int8/16/32 AVX2) + Graph + re-rank multi-fator + meaning-first
```

Camadas que definem o projeto:

- **Governança/Judicial:** internal/proposal (Kernel Isolado, fail-closed), internal/gate, internal/decision (event-sourced), internal/cognitive (CV ledger), internal/deliberate (conselho determinístico).
- **Segurança:** internal/auth (JWT), internal/integrity (Family Chain Ed25519), internal/hardening (PreMain), internal/secrets (vault), internal/quarantine (zona epistemológica), internal/contenttrust, pkg/cosca/jail (auto-jail Bubblewrap).
- **Knowledge/Memória:** internal/knowledge (Engine + compilador), internal/memory (6 camadas), internal/search, internal/vector (SQLiteVec + AVX2), internal/embeddings, internal/indexer, internal/chunker, internal/grounding (RAG grounded), internal/ranking, internal/semantic_discovery.
- **Orquestração:** internal/orchestration (pipeline de agents MAG), internal/orchestrator (TaskOrchestrator), internal/dflow (workflow determinístico), internal/workflow (rotas tipadas), internal/agents, internal/agentbridge (supervisionar agents externos).
- **Living World:** internal/world (World Model), internal/worldmodel, internal/scene, internal/nodegraph (ComfyUI-style), internal/gameengine (ECS), internal/procgen (splitmix64 determinístico), internal/render, internal/vision (tesseract), internal/voice (TTS concatenativa).
- **API:** api/rest (94 endpoints), api/grpc (12 RPCs), api/auth, api/middleware, api/stream.

3.2 Hermes Agent — 'agente de produção'

O Hermes é um **agente conversacional com learning loop**, empacotado como um serviço com múltiplas frentes de entrada (gateway). Um core de agente em Python conversa com 3 superfícies de UI e dezenas de plataformas de chat.

```
run_agent.py / agent/ -> loop: contexto -> modelo -> tools -> streaming
tools/ (registry) + toolsets.py -> ~centenas de tools em toolsets
CRON (scheduler) + skills/ (agentskills.io) + delegate_tool (subagentes)
gateway/platforms/ + plugins/platforms/ -> Telegram, Discord, Slack, WA, Signal, Matrix, IRC, etc.
tui_gateway JSON-RPC 2.0 -> ui-tui (Ink) | apps/desktop (Electron) | web/ (React/Vite)
```

Camadas que definem o projeto:

- **Loop do agente:** run_agent.py (AI-Agent), agent/conversation_loop.py, agent/agent_init.py, agent/turn_finalizer.py.
- **Tools:** tools/registry.py (ToolRegistry singleton), toolsets.py (TOOLSETS + _HERMES_CORE_TOOLS), model_tools.py (schema para o modelo), tool_dispatch_helpers (segment planner).
- **Learning loop:** agent/background_review.py (fork do agente após o turno), agent/curator.py, agent/learn_prompt.py, agent/learning_graph.py, agent/skill_manager_tool.py. Nudges a cada N turnos.
- **Memória:** hermes_state*.py (SQLite FTS5), tools/memory_tool.py (MEMORY.md/USER.md), tools/session_search_tool.py (recall longo), plugins/memory/honcho (modelo de usuário).
- **Providers:** providers/base.py (ProviderProfile dataclass) + plugins/model-providers/ (37) + providers/__init__.py (registry).
- **Backends de execução:** tools/environments/ (BaseEnvironment ABC) — local, Docker, SSH, Singularity, Modal, Daytona, Vercel Sandbox.
- **UI/Gateway:** gateway/run.py, gateway/platforms/base.py (BasePlatformAdapter), tui_gateway (JSON-RPC), relay/ (apps), ui-tui, apps/desktop, web/.

4. Comparação Código-a-Código (tabela de mapeamento)

Mapeamento direto conceito->arquivo dos dois lados. Quando um lado não tem equivalente, a célula indica — (não possui). Isso revela onde cada sistema é realmente diferente.

Dimensão	Cosca (arquivo/conceito)	Hermes Agent (arquivo/conceito)	Nota igualdade/divergência
Loop de agente	internal/orchestration (Engine.Execute + executor.go) — pipeline MAG determinístico	run_agent.py / agent/conversation_loop.py (while + tool-calls + streaming)	Cosca: pipeline tipado copy-on-write com watchdog anti-stall. Hermes: loop discreto por turno com fork p/ review.
Registro de tools	internal/skills + internal/skillevail + internal/tools	tools/registry.py (ToolRegistry) + toolsets.py + tools/registry.register()	Hermes tem registro dinâmico por decorator + schema p/ modelo. Cosca tem catalogo de skills/engines.
Schema LLM	internal/providers + internal/models/modelreg	model_tools.py get_tool_definitions() + ProviderProfile (providers/base.py)	Hermes gera schema para tools; Cosca resolve provedores/modelos nativamente.
Memória persistente	internal/memory (6 camadas + TTL) + hermes_state.py (SQLite+WAL)	hermes_state*.py (SessionDB SQLite + FTS5)	Cosca: camadas de escopo + RBAC + blockchain/Merkle. Hermes: sessões + memória curada.
Memória curada	internal/memory/curated.go (sanitização de falhas)	tools/memory_tool.py (MEMORY.md/USER.md) + nudges	Hermes tem MEMORY.md/USER.md explícitos; Cosca tem curadoria automatizada de falhas.
Learning loop	internal/memory/register.go (blockchain de aprendizados + Merkle) + evolution.md	agent/background_review.py + curator.py + learn_prompt.py (cria skills da experiência)	Hermes cria/melhora skills automaticamente; Cosca registra aprendizado com proveniência + hash encadeado.
Busca semântica	internal/search (híbrida FTS5+vivec+grafo+re-rank) + internal/knowledge	hermes_state_search.py (FTS5 + trigram + CJK + like fallback) + session_search_tool	Cosca: pipeline multi-fator com meaning-first. Hermes: recall de sessões passadas.
Providers LLM	internal/providers (catálogo nativo + registry dinâmico + circuit breaker + rate limit)	providers/base.py + plugins/model-providers/ (37) + runtime_provider.py	Hermes tem muito mais providers prontos e troca sem lock-in por profile. Cosca tem resiliência própria (breaker/ratelimit).
Backends de execução	pkg/cosca/jail (auto-jail Bubblewrap) + plugins sandbox (seccomp/Rlimit)	tools/environments/ (BaseEnvironment ABC) — 7 backends (local, docker, ssh, singularity, modal, daytona, vercel)	Hermes roda TUDO em qualquer lugar. Cosca coloca o próprio binário na jaula (segurança).
Sandboxing	pkg/cosca/jail.go (execute copia em RAM + Bubblewrap, fail-closed) + sandbox_wrapper.go (seccomp)	tools/environments/docker.py (cap-drop ALL) / singularity.py / modal.py + _ThreadedProcessHandle	Cosca: auto-jail por padrão, recusa root. Hermes: sandbox por container/cloud configurável.
Segurança de conteúdo	internal/contenttrust (envelope cosca-untrusted-data-v1) + quarantine	agent/tool_guardrails.py (detecção de prompt-injection em tools)	Cosca trata todo texto externo como não confiável; Hermes guarda tools específicas.
Auditoria/proveniência	internal/audit + internal/ledger (CADERNO P1-P8) + internal/provenance + internal/trace	— (logs de sessão, sem ledger criptográfico de auditoria)	Grande diferencial do Cosca: trilha de auditoria + ledger tamper-evidente + proveniência científica.

Dimensão	Cosca (arquivo/conceito)	Hermes Agent (arquivo/conceito)	Nota igualdade/divergência
Workflows	internal/dflow (replay determinístico, estilo Temporal) + internal/workflow (rotas tipadas)	cron/scheduler.py (cronjob + parse_schedule) + scripts/	Cosca: workflows duráveis determinísticos. Hermes: cron agendado em linguagem natural.
Auto-consolidação de conhecimento	internal/knowledge (compilador, aging, backfill, semantic_discovery, epistemic)	agent/learning_graph.py (grafo de aprendizado) + curator.py (dedup de skills)	Cosca tem epistemologia FAC T/MEASURED/EVIDENCE/...; Hermes tem grafo de aprendizado visível.
Interface principal	Cobra CLI (97 cmd) + REST (94) + gRPC (12) + SDK em Go	CLI hermes + TUI Ink + Web (React) + Desktop (Electron) + JSON-RPC + MCP + gateway messaging	Hermes tem muito mais superfícies prontas para o usuário final.
Mensageria (Telegram/Discord/etc.)	— (não possui gateway de messaging)	gateway/platforms/ + plugins/platforms/ (22+) — Telegram, Discord, Slack, WhatsApp, Signal, Matrix, IRC...	Hermes é ubíquo; Cosca não tem esse plano.
Provisionadores de runtime cloud	— (roda on-prem/WASM)	Modal, Daytona, Vercel Sandbox + Docker/SSH/Singularity	Hermes: serverless com hibernação. Cosca: self-contained local.
Mundo 3D / percepção espacial	world/worldmodel/scene/nodegraph/gameengine/procgen/render + vision + voice	— (não possui; foco em texto/tools)	Unicidade do Cosca: Living World determinístico com percepção e geração procedural.
Modelo de usuário	— (RBAC por papel; sem perfil psicológico)	plugins/memory/honcho (dialética, peer cards, conclusões)	Hermes constrói modelo de quem você é ao longo das sessões.
Auto-jail / machine-bound	pkg/cosca/jail + internal/integrity (DPAPI/chain) + internal/license (3 fatores)	— (open source, sem cadeia de confiança do binário)	Cosca: o próprio binário é a jaula; licença por 3 fatores.

5. Matriz de Capacidades — O que cada um consegue fazer

Avaliação por capacidade: [sim] nativo, ■? parcial, — não possui. Serve para responder rápido 'meu sistema faz isso?'.

Capacidade	Cosca	Hermes
Orquestração de múltiplos agentes hierárquicos	[sim] (Don->Kernel->CEOs->Chiefs->Specialists)	■? (subagentes via delegate_task, sem hierarquia)
Delegação com paralelismo isolado	■? (chain/ExecuteParallel)	[sim] (delegate_tool + DaemonThreadPool + async_delegation)
Loop self-improving (cria skills sozinho)	■? (registro de aprendizado + evolution.md)	[sim] (background_review + curator + nudges)
Memória semântica de longo prazo	[sim] (6 camadas + FTS5 + embeddings + RBAC)	[sim] (FTS5 + MEMORY.md/USER.md + Honcho)
Busca híbrida (vetal + grafo + re-rank)	[sim] (int8/16/32 AVX2, meaning-first)	■? (FTS5 + recall de sessões)
RAG com fidelidade verificável (grounded)	[sim] (grounding/ + RecallGate + QRELS)	■? (sem gate de fidelidade)
Roteamento semântico de queries	[sim] (semantic_router >=0.80 + margem)	■? (rota por plataforma/ferramenta)
37+ LLM providers sem lock-in	■? (~12 nativos)	[sim] (37 plugins + profile)
Rodar em qualquer lugar (VPS/cloud/serverless)	■? (on-prem/WASM/jail)	[sim] (7 backends, serverless hibernante)
Interfaces para o usuário (Web/Desktop/TUI)	■? (CLI + REST + gRPC)	[sim] (TUI Ink + Electron + React)
Messaging (Telegram/Discord/WhatsApp/Signal)	—	[sim] (gateway 22+ plataformas)
Cron em linguagem natural	■? (cron + workflows)	[sim] (cronjob natural + entrega multi-plataforma)
Mundo 3D / Living World determinístico	[sim] (world/worldmodel/procgen/escena/vfx)	—
Sandboxing/segurança fail-closed	[sim] (auto-jail + seccomp + contenttrust + quarantine)	■? (containers/cloud)
Auditoria criptográfica + proveniência	[sim] (ledger P1-P8 + trace + provenance)	—
Workflows duráveis determinísticos	[sim] (dflow replay)	■? (cron)
SDK/API programática	[sim] (SDK Go + REST + gRPC)	[sim] (JSON-RPC + REST + MCP)
Skills padrão aberto (agentskills.io)	■? (formato próprio)	[sim] (200 skills compatíveis)
OCR/visão local	[sim] (tesseract local)	■? (vision via provider)
TTS/voz local determinístico	[sim] (concatenativo por dífonos)	■? (streaming TTS no gateway)

Leitura rápida: o **Cosca é um sistema de cognição e governo** (conhece, audita, restringe, decide). O **Hermes é um agente de interfacen e autonomia** (fala, roda, evolui, está em todo lugar).

6. Vantagens do Cosca

Lista dos diferenciais mais fortes do Cosca, com os arquivos que os sustentam.

6.1 Governança determinística — 'a IA propõe, o sistema decide' (P9)

O princípio central de todo o código: **o LLM nunca decide sozinho**. Toda a aritmética de gate (métricas de convergência, confiança, veredicto) é computada em Go puro, sem rede. Isso torna o sistema **testável com go test** sem preciso de um modelo. Onde o Hermes delega o julgamento ao LLM, o Cosca coloca uma camada determinística entre a IA e a ação.

```
internal/deliberate/convergence.go: ComputeConvergence (ZERO-LLM, S peso×concordância)
internal/deliberate/confidence.go: ComputeConfidence (base + penalidades tipadas por critic/risco)
internal/proposal/flow.go: veredicto fail-closed + hash binding + single-use
```

6.2 Fail-closed absoluto e 'Kernel Isolado'

O Cosca trata **tudo que vem de fora como não confiável até provar inocência**. Sem veredicto, não há execução. A proposta é selada (cópia imutável) antes da validação e o hash é checado antes e depois — se o artefato divergir durante o julgamento, é DENY. Autorização single-use (sem replay) e laço REVIEW limitado a 2 revisões.

```
internal/proposal/flow.go:126 -> se ProposalHash diverge após validação: DENY fail-closed
internal/proposal/semantic.go: -> OllamaValidator: só aceita 'VALIDAR' ou 'REJEITAR:'
-> qualquer sufixo/ambiguidade/vazio = DENY
```

6.3 Segurança de ponta: auto-jail, Family Chain, contenttrust

O próprio binário é a jaula (pkg/cosca/jail): se não está jail, se re-forka em cópia RAM via Bubblewrap, rejeita rodar como root e fallback é **negar execução** (não avisar). A Family Chain (internal/integrity) usa assinatura Ed25519 + git-anchor para bloquear o boot se a cadeia quebrar. Segredos nunca atravessam a fronteira via argv. Nenhum desses mecanismos existe no Hermes (que é open source e roda como processo normal).

6.4 Auditoria e proveniência criptográfica

É a estrutura de confiança mais madura dos dois. O **ledger** (internal/ledger) implementa 8 propriedades (P1-P8): WAL com fsync, MVCC com CAS, append-only com tombstone, cadeia de hash SHA-256. A **proveniência** distingue ClaimKind (observed/calculated/simulated/generated/hypothesis) — 'nunca inventar resultados'. O **trace** carrega TraceID + hash de entrada/saída + versão cognitiva, com detecção determinística de divergência. Nada disso existe no Hermes.

6.5 Memória semântica curada com verdade epistêmica

O Cosca tem um **modelo de verdade**: classes epistemológicas FACT/MEASURED/EVIDENCE/INFERRED/RULE/DECISION/PROFILE com filtro fail-closed; quarentena epistemológica (o que a IA inventa não entra direto); grounding com verificação de claims por token-overlap + RecallGate de não-regressão + QRELS congelado. A memória é em 6 camadas com TTL e RBAC por papel (Don/Kernel/Chief/Specialist). Busca híbrida FTS5 + vetores int8/16/32 com kernels AVX2 + grafo + re-rank multi-fator (BM25 0.25 + vetor 0.35 + grafo 0.20 + frescor 0.10 + popularidade 0.10) + meaning-first.

6.6 World Model 3D determinístico (Living World)

Unicidade absoluta. Um mundo canônico em Go puro (internal/world): mesmas sementes + dados + versão = mesmo mundo. Node graph estilo ComfyUI com cache por assinatura recursiva; ECS (gameengine); procgen determinístico portátil (splitmix64); render com checkpoint por nó; percepção via vision (tesseract local) e voz concatenativa por dífonos. O Hermes não tem nada disso — é um agente de texto/tools.

6.7 Workflows duráveis determinísticos

internal/dflow é um padrão estilo Temporal: workflow é função pura/determinística (sem clock/RNG//O) e as activities têm retry idempotente por histórico. SleepUntil é durável; HITL via WaitForEvent/ResolveEvent. O Hermes usa cron agendado, sem replay determinístico.

6.8 Binário único, self-contained, com SDK

Go compila para um binário estático com SQLite embutido (modernc.org/sqlite — sem CGO), WASM via wazero. Roda on-prem sem dependências externas. Expõe SDK Go, REST (94 endpoints) e gRPC (12 RPCs) com RBAC fail-closed (jwtSecret vazio ⇒ rejeita rotas protegidas) e roteamento modular (modlink) que confina a busca por

escopo.

7. Vantagens do Hermes Agent

7.1 Self-improving loop — o maior diferencial

O Hermes é 'o agente que melhora sozinho'. Após cada turno, o finalizador faz um **fork** do agente (novo AI Agent, ~30k tokens) em background e o instrui a **criar/editar skills** ou **salvar memória**. O prompt do curador força proatividade: 'most sessions produce at least one skill update'. Preferência de edição: patch em skill carregada -> umbrella existente -> suport file -> nova umbrella. Há **read-before-write enforced** e proteção de skills bundled/hub/pinned. Nudges a cada N turnos (memory=10, skill=2) disparam o review.

```
agent/turn_finalizer.py:800 -> if should_review_memory or should_review_skills: _spawn_background_review(...)
agent/background_review.py:476 -> _SKILL_REVIEW_PROMPT (cria/edita skills, preferência de patch)
agent/skill_manager_tool.py -> skill_manage (criar/editar/validar skills por frontmatter)
```

7.2 Ubiquidade — fala com você onde você está

Um único gateway (gateway/run.py) conecta **20+ plataformas**: Telegram, Discord, Slack, WhatsApp, Signal, Matrix, IRC, Line, Email, SMS/Twilio, Teams, Mattermost, DingTalk, Feishu, WeCom, Google Chat, Home Assistant, ntfy, Simplex. Mais as interfaces Desktop (Electron), Web (React/Vite), TUI (Ink). Tudo fala JSON-RPC 2.0 com o backend Python. O Cosca não tem nenhuma dessas superfícies prontas — só CLI/REST/gRPC.

7.3 37 LLM providers sem lock-in

O sistema de providers é **declarativo e plugins-pud**: cada provider é um `ProviderProfile` (dataclass) registrado dinamicamente (bundled + \$HERMES_HOME + entry-point pip + legacy). O transporte monta os kwargs a partir do profile, não de flags, e a resolução de runtime é feita por `api_mode` + detecção de hostname/URL + `fetch_models()` ao vivo. Trocar de modelo é ``hermes model`` — sem mudar código. Cobre Nous, OpenAI, Anthropic, Gemini, Bedrock, OpenRouter, GitHub Copilot, DeepSeek, xAI, NVIDIA, Meta, Alibaba, e muitos outros.

7.4 Roda em qualquer lugar — 7 backends de execução

Uma única ABC ``BaseEnvironment`` (`_run_bash` + `cleanup`) com 7 implementações: local, Docker, SSH, Singularity, `**Modal**`, `**Daytona**`, `**Vercel Sandbox**`. Modal e Daytona oferecem **serverless com hibernação** (o ambiente dorme quando ocioso e acorda sob demanda, custando quase nada). Rode num VPS de \$5 ou num cluster GPU. O Cosca roda on-prem self-contained, mas não provisiona cloud.

7.5 Skills prontas no padrão aberto agentskills.io

`**200 skills**` (82 padrão + 118 optional) em formato agentskills.io (frontmatter YAML + references/templates/assets/scripts). Progressive disclosure: ``skills_list`` (só metadata) e ``skill_view`` (carrega conteúdo sob demanda). Gating por plataforma e ambiente. Compatível com o hub de skills. O Cosca usa um formato próprio com 29 skills em disco.

7.6 Delegação e paralelismo maduros

``delegate_task`` cria subagentes isolados (contexto novo, `task_id` próprio, toolsets herdados com blocklist). Paralelização via `DaemonThreadPoolExecutor` com ``contextvars.copy_context()``, polling não-bloqueante e propagação de interrupt. Fan-out assíncrono. Nested delegation só por role orchestrator. Isolamento de contexto via `ContextVar`.

7.7 Auto-consolidação de conhecimento + modelo de usuário

O ``learning_graph`` torna o aprendizado 'visível' (memórias ligadas a skills); o ``curator`` deduplica skills. O plugin Honcho constrói um **modelo dialético do usuário** (peer cards, raciocínio, conclusões) — o agente desenvolve uma compreensão crescente de quem você é. Busca em conversas passadas com resolução de linhagem de sessão.

7.8 Ferramentas para pesquisa e treino de modelos

``batch_runner.py`` gera trajetórias em lote; ``trajectory_compressor.py`` comprime trajetórias com LLM para treinar a próxima geração de modelos tool-calling. Serve tanto de agente quanto de pipeline de coleta/compressão de dados.

7.9 Maturidade de pipeline de tools e RPC

O sistema de tools é o mais completo dos dois: registry singleton com override por plugin, descoberta automática por AST scan, schema produzido para o modelo, dispatch assíncrono, e um **servidor JSON-RPC 2.0** (tui_gateway) com ~130 handlers. Scripts Python podem chamar tools via RPC, colapsando pipelines multi-etapas em turns de

custo-zero de contexto.

8. Onde cada um perde (gaps honestos)

8.1 Fraquezas do Cosca

- **Sem superfícies prontas para o usuário final.** Não tem TUI rica, Web app, Desktop, nem gateway de messaging (Telegram/Discord/WhatsApp). É acessível só via CLI/REST/gRPC.
- **Menos providers e menor portabilidade.** ~12 providers nativos vs 37 do Hermes; não há execução em Modal/Daytona/Vercel/serverless.
- **Sem self-improving automático de skills.** O Cosca registra aprendizado com proveniência, mas não cria/edita skills sozinho na mesma medida — a autonomia de evolução é menor.
- **Living World parcialmente stub.** Os pipelines de worldmodel (processVision/processSpatial/processAudio/coordinateAgents) são declarados mas a execução está por implementar — é esqueleto.
- **Contagem interna defasada.** README anuncia 53 agents/28 skills; disco tem 61/29 — badges desatualizados. Duas camadas de 'orquestrador' (internal/orchestration vs internal/orchestrator) podem confundir.
- **Pesos.** 846MB de fonte, ~25.500 arquivos .go — um monólito grande para manter, com risco de duplicação entre KDFs (HKDF vs SHA-256 iterado) e entre mecanismos de integridade.
- **Sem modelo de usuário.** RBAC por papel, mas não constrói perfil psicológico crescente do Don (como o Honcho do Hermes).

8.2 Fraquezas do Hermes Agent

- **Sem governança determinística.** O julgamento é delegado ao LLM. Não há gate de veredicto testável sem modelo; a confiança não é calculada em código puro.
- **Sem auditoria criptográfica/proveniência.** Não tem ledger tamper-evidente, nem Family Chain, nem proveniência científica de claims. A 'verdade' é o que o LLM guarda.
- **Sem mundo 3D / percepção espacial / Living World.** É estritamente agente de texto e tools.
- **Segurança menos agressiva.** Roda como processo normal; sandbox é por container/cloud configurável. Não tem auto-jail por padrão, nem rejeição a root, nem cadeia de confiança do binário.
- **Escala de tooling pode gerar caos.** Toolsets com dezenas de tool call para um único modelo — risco de custo/latência; o próprio mecanismo de 'fork de 30k tokens' para review por turno é caro e pode ser desperdício para tarefas simples.
- **Sem trilha forense de execução.** Não há traces causais com hash de entrada/saída nem flight recorder para reconstruir 'por que o agente fez isso'.
- **Dependências pesadas.** Python 3.11 + Node.js + Nix + Docker para rodar; mais frágil de instalar que um binário único Go.

9. Veredicto e Recomendação para o Don

Dimensão	Vencedor	Justificativa curta
Governança e segurança	Cosca	Zero-LLM, fail-closed, auto-jail, Family Chain, quarantine, contenttrust
Memória e busca semântica	Empate	Cosca: epistemologia + grounding + grafo. Hermes: recall de sessões + modelagem de usuário
Aprendizado autônomo	Hermes	Cria/melhora skills sozinho (self-improving loop real)
Portabilidade/Ubiquidade	Hermes	7 backends + 37 providers + Telegram/Discord/Web/Desktop
Auditoria/proveniência	Cosca	Ledger P1-P8, trace causal, proveniência por ClaimKind
Mundo 3D / percepção	Cosca	Living World determinístico (nenhum no Hermes)
Prontidão como produto	Hermes	Interface rica + skills prontas + cron multi-plataforma
Eficiência operacional (ops)	Cosca	Binário único Go, SQLite embutido, atômico

Conclusão executiva: o Cosca é um **cérebro corporativo** — pensou em quem manda, em segurança, em verdade auditável e em um mundo 3D próprio. O Hermes é um **agente de produção** — pensou em estar em todo lugar, evoluir sozinho e entregar valor imediato ao usuário. Eles são **ortogonais**. O Cosca não vira Hermes e o Hermes não vira Cosca; a questão é qual você prioriza, e se um pode alimentar o outro.

9.1 O que o Cosca pode aprender do Hermes

- **Self-improving de skills:** adotar um loop de background_review que cria/edita skills automaticamente após tarefas complexas (com proveniência no ledger do Cosca).
- **Modelo de usuário:** incorporar algo como o Honcho (peer cards dialéticos) para construir um perfil crescente do Don.
- **Camada de interfaces:** adicionar TUI/Web/Desktop e um gateway de messaging como consumidores do REST/gRPC já existentes.
- **Mais providers declarativos:** migrar para o padrão ProviderProfile/registry plugins-pud para os ~12 nativos.
- **Backends cloud opcionais:** plugar Modal/Daytona/Vercel como ambientes de execução alternativos do terminal, mantendo a auto-jail como padrão on-prem.
- **Padrão aberto de skills:** considerar compatibilidade agentskills.io para reaproveitar as 200 skills existentes.

9.2 O que o Hermes pode aprender do Cosca

- **Camada determinística de veredicto:** um gate de confiança em código puro entre a resposta do LLM e a ação (como internal/proposal).
- **Auditoria/proveniência:** um ledger append-only criptográfico para registrar o 'porquê' das ações e reconstruí-las.
- **Modelo de verdade epistêmica:** classes FACT/MEASURED/EVIDENCE/INFERRED para evitar que a IA apresente hipótese como fato.
- **Fail-closed de conteúdo:** tratar texto externo como não confiável (contenttrust) para reduzir prompt-injection.
- **Segurança de execução:** sandbox por padrão (como auto-jail) com rejeição implícita a rodar como root.

Recomendação ao Don: dado que o Cosca é a prioridade da família (v1.5.0), sugiro **importar do Hermes** as capacidades de *self-improving de skills* (com proveniência no ledger) e a *camada de modelagem de usuário*, que são as duas maiores lacunas do Cosca em autonomia e personalização — sem abrir mão da governança determinística que é o coração do projeto. O Hermes serve de benchmarking de produto; o Cosca continua sendo o cérebro.

10. Apêndice — Metodologia e Fontes

Objetivo: comparar em detalhe o Cosca e o Hermes Agent, código a código e capacidade a capacidade, produzindo um PDF robusto com as vantagens de cada um.

Fontes

- **Hermes Agent:** clone oficial do branch principal — github.com/nousresearch/hermes-agent (commit de referência). 10.528 arquivos.
- **Cosca:** repositório da família — github.com/CoscaAI/cosca, versão v1.5.0, branch main, monólito Go.

Método

- Downloads/medição direta no código (contagem de arquivos, comandos, endpoints, providers, skills).
- Análise arquitetural por camadas: delegação a agentes de exploração profunda (núcleo, providers/backends, apps/skills do Hermes; cognição/orquestração, memória/knowledge e infra/living world do Cosca).
- Mapeamento de conceito->arquivo (tabela de código-a-código) e matriz de capacidades.
- Veredicto por dimensão com justificativa; recomendação orientada a prioridade da família.

Limitações

As contagens de arquivos/comandos/endpoints são medidas no momento da análise e podem variar com o tempo; o Cosca tem README desatualizado em relação ao disco. Os pipelines de Living World do Cosca e alguns recursos do Hermes podem estar em estado parcial (stub). A análise é técnica/estrutural e não mede desempenho em runtime nem qualidade de saída de modelo.

Documento CONFIDENCIAL para uso interno da família Cosca. Elaborado pelo Consigliere (Cosca Kernel).